



数据库原理

实验报告

学 号： 202411221

姓 名： 王程远

提交日期： 2026-4-23

成 绩：

东北大学秦皇岛分校计算机与通信工程学院

2026 年 4 月 23 日



【实验内容】

【实验一】

问题 1:

在查询分析器中创建一个数据库，要求如下：

- (1) 数据库名称 Test2。
- (2) 主要数据文件：逻辑文件名为 Test2_data1，物理文件名为 Test2_data1.mdf，初始容量为 10MB，最大容量为 100MB，增幅为 10MB。
- (3) 次要数据文件：逻辑文件名为 Test2_data2，物理文件名为 Test2_data2.ndf，初始容量为 10MB，最大容量为 10MB，增幅为 15%。
- (4) 事务日志文件：逻辑文件名为 Test2_log1，物理文件名为 Test2_log1.ldf，初始容量为 10MB，最大容量为 50MB，增幅为 20MB。

实现代码：

对象 | 无标题 (SQL Server 实验课)

SQL Server 实验课

```
1 CREATE database Test2 ON PRIMARY(  
2     name = Test2_data1,  
3     filename = '/var/opt/mssql/data/Test2_data1.mdf',  
4     size = 10mb,  
5     maxsize = 100mb,  
6     filegrowth = 10mb  
7 ),  
8 (  
9     name = Test_data2,  
10    filename = '/var/opt/mssql/data/Test2_data2.ndf',  
11    size = 10mb,  
12    maxsize = 10mb,  
13    filegrowth = 15%  
14 )  
15 LOG ON(  
16     name = Test2_log1,  
17     filename = '/var/opt/mssql/data/Test2_log1.ldf',  
18     size = 10mb,  
19     maxsize = 50mb,  
20     filegrowth = 20mb  
21 );
```

**问题 2:**

在查询分析器中按照下列要求修改第 3 题中创建的数据库 test2

- (1) 主要数据文件的容量为 50MB，最大容量为 200MB，增幅为 20MB。
- (2) 次要数据文件的容量为 50MB，最大容量为 300MB，增幅为 20MB。
- (3) 事务日志文件的容量为 30MB，最大容量为 100MB，增幅为 10%。

实现代码:

```
1  ALTER database Test2
2  --MODIFY FILE(
3      name = Test2_data1,
4      size = 50mb,
5      maxsize = 200mb,
6      filegrowth = 20mb);
7
8  ALTER database Test2
9  --MODIFY FILE(
10     name = Test2_data2,
11     size = 50mb,
12     maxsize = 300mb,
13     filegrowth = 20mb);
14
15  ALTER database Test2
16  --MODIFY FILE(
17     name = Test2_log1,
18     size = 30mb,
19     maxsize = 100mb,
20     filegrowth = 10%);
```

问题 3:

数据库更名: 把 test1 数据库更名为 new_test1

实现代码:

```
1  EXEC sp_renamedb 'Test1', 'new_Test1';
```

问题 4:

在企业管理器中删除 new_test1 数据库，在查询分析器中删除 test2 数据库。

实现代码:

```
1  DROP DATABASE Test2;
```

**【实验二】****问题一：**

创建数据库 studentInfo，包含如下表，创建这些表并按要求定义约束：

表 2.1 student（学生表）结构

字段名	说明	数据类型	约束说明
Student_id	学号	字符串，长度为 10	主键
Student_name	姓名	字符串，长度为 10	非空
sex	性别	字符串，长度为 1	非空值，取 ‘F’ 或 ‘M’
age	年龄	整数	允许空值
department	所在系名	字符串，长度为 15	默认值为 ‘computer’

表 2.2 course（课程表）结构

字段名	说明	数据类型	约束说明
Course_id	课程号	字符串，长度为 6	主键
Course_name	课程名	字符串，长度为 20	非空值
PreCouId	先修课程号	字符串，长度为 6	允许空值
Credits	学分	十进制数，精度 3，小数位 1	非空值

表 2.3 score（选课表）结构

字段名	说明	数据类型	约束说明
Student_id	学号	字符串，长度为 10	外键，参照 student 的主键
Course_id	课程号	字符串，长度为 6	外键，参照 course 的主键
Grade	成绩	十进制数，精度 3，小数位 1	允许空值，要求 >0 <100
联合主键：（Student_id，Course_id）			

以下为各个表的数据：

Students 表数据

Student_id	Student_name	sex	age	department
20010101	Jone	M	19	Computer
20010102	Sue	F	20	Computer
20010103	Smith	M	19	Math
20030101	Allen	M	18	Automation



20030102	deepa	F	21	Art
20010104	Stefen	F	20	Computer

Course 表数据

Course_id	Course_name	PreCouId	Credits
C1	English		4
C2	Math	C1	2
C3	Cprogram	C2	2
C4	database	C2	2

Score 表数据

Student_id	Course_id	Grade
20010101	C1	90
20010102	C1	87
20010103	C1	88
20010102	C2	90
20010104	C2	94
20010102	C3	62
20030101	C3	80
20010103	C4	77



实现代码:

创建数据库

```
01 CREATE DATABASE studentInfo
02 on PRIMARY
03 (
04     name = 'studentInfo_data1',
05     filename = '/var/opt/mssql/data/studentInfo_data1.mdf',
06     size = 50mb,
07     maxsize = 500mb,
08     filegrowth = 10mb
09 ),
10 (
11     name = 'studentInfo_data2',
12     filename = '/var/opt/mssql/data/studentInfo_data2.ndf',
13     size = 50mb,
14     maxsize = 500mb,
15     filegrowth = 10mb
16 )
17
18 log ON
19 (
20     name = 'studentInfo_log1',
21     filename = '/var/opt/mssql/data/studentInfo_log1.ldf',
22     size = 20mb,
23     maxsize = 200mb,
24     filegrowth = 10mb
25 );
```



创建表

```
01  UE studentInfo;
02  GO
03
04  CREATE TABLE student (
05      Student_id CHAR(10) PRIMARY KEY,
06      Student_name CHAR(10) NOT NULL,
07      sex CHAR(1) NOT NULL CHECK(sex IN ('F', 'M')),
08      age INT NULL,
09      department CHAR(15) DEFAULT 'computer'
10  );
11
12  create table course (
13      Course_id char(6) primary key,
14      Course_name char(20) NOT NULL,
15      PreCouId char(6) NULL,
16      Credits numeric(3, 1) NOT NULL
17  );
18
19  create table score (
20      Student_id char(10),
21      Course_id char(6),
22      Grade numeric(3, 1) NULL check(Grade > 0 and Grade < 100),
23      primary key(Student_id, Course_id),
24      foreign key(Student_id) references student(Student_id),
25      foreign key(Course_id) references course(Course_id)
26  );
```

问题二:

增加、修改、删除字段，要求：

- (1) 为表 student 增加一个 memo（备注）字段，类型为 varchar（200）。
- (2) 将 memo 字段的数据类型更改为 varchar（300）。
- (3) 删除 memo 字段



实现代码:

```
01  -- (1)  为表 student 增加一个 memo (备注) 字段, 类型为 varchar (200) 。
02  alter table student add memo varchar(200);
03  GO
04
05  -- (2)  将 memo 字段的数据类型更改为 varchar (300) 。
06  alter table student alter column memo varchar(300);
07  GO
08
09  -- (3)  删除 memo 字段
10  alter table student drop column memo;
11  GO
```

问题三:

向表中插入数据验证约束



实现代码:

```
01 USE studentInfo;
02 GO
03
04 -----
05 -- 1. 验证 DEFAULT 约束 (默认值)
06 -----
07 -- 我们不输入 department 字段的值。
08 -- 插入成功后, 因为默认约束, Alice 的 department 会自动变成 'computer'。
09 INSERT INTO student (Student_id, Student_name, sex, age)
10 VALUES ('S001', 'Alice', 'F', 20);
11
12 -- 插入一条合法课程数据, 为下面做准备
13 INSERT INTO course (Course_id, Course_name, Credits)
14 VALUES ('C001', 'Database', 3.0);
15
16
17 -----
18 -- 2. 验证 CHECK 约束 (检查约束)
19 -----
20 -- 预期报错: 检查性别。因为性别被限制为了 'F' 或 'M', 输入 'X' 会报错拦截。
21 INSERT INTO student (Student_id, Student_name, sex, age, department)
22 VALUES ('S002', 'Bob', 'X', 21, 'Math');
23
24 -- 预期报错: 检查分数。因为 Grade 限制了  $0 < Grade < 100$ , 输入 120 会报错拦截。
25 INSERT INTO score (Student_id, Course_id, Grade)
26 VALUES ('S001', 'C001', 120);
27
28
29 -----
30 -- 3. 验证 PRIMARY KEY 约束 (主键唯一)
31 -----
32 -- 预期报错: 上面已经插入过学号为 'S001' 的学生, 学号是主键不能重复。
33 INSERT INTO student (Student_id, Student_name, sex)
34 VALUES ('S001', 'Charlie', 'M');
35
36
37 -----
38 -- 4. 验证 FOREIGN KEY 约束 (外键约束)
39 -----
40 -- 预期报错: 学号 'S999' 根本不存在于 student 表中, 不能直接给其录入成绩。
41 INSERT INTO score (Student_id, Course_id, Grade)
42 VALUES ('S999', 'C001', 80);
```



运行结果

```
01 开始执行查询的位置 行 3
02 (1 行受到影响)
03 (1 行受到影响)
04 Msg 547, Level 16, State 0, Line 21
05 The INSERT statement conflicted with the CHECK constraint
06 "CK__student__sex__37A5467C". The conflict occurred in database "studentInfo",
07 table "dbo.student", column 'sex'.
08 The statement has been terminated.
09 Msg 8115, Level 16, State 8, Line 25
10 Arithmetic overflow error converting int to data type numeric.
11 The statement has been terminated.
12 Msg 2627, Level 14, State 1, Line 33
13 Violation of PRIMARY KEY constraint 'PK__student__A2F7EDF49F2D8CB0'. Cannot
14 insert duplicate key in object 'dbo.student'. The duplicate key value is
15 (S001 ).
16 The statement has been terminated.
Msg 547, Level 16, State 0, Line 41
The INSERT statement conflicted with the FOREIGN KEY constraint
"FK__score__Student_i__3E52440B". The conflict occurred in database
"studentInfo", table "dbo.student", column 'Student_id'.
The statement has been terminated.
总执行时间: 00:00:00.041
```

问题四:

分别使用企业管理器和查询分析器删除表

实现代码:

```
1 use studentInfo;
2 GO
3
4 drop table Score, student, course;
```

**【实验三】**

在已经建立的 studentinfo 数据库和 3 个 students、courses、score 基础上完成下列操作：

问题一：

向 students 表添加一个学生记录，学号为 20010112，性别为男，姓名为 stefen，年龄 25 岁，所在系为艺术系 art。

实现代码：

```
1 Use studentInfo;
2 GO
3
4 Insert Into student(Student_id, Student_name, sex, age, department)
5 Values ('20010112', 'stefen', 'm', 25, 'art');
```

问题二：

向 score 表添加一个选课记录，学生学号为 20010112，所选课程号为 C2。

实现代码：

```
1 Use studentInfo;
2 Go
3
4 Insert Into score(Student_id, Course_id)
5 Values ('20010112', 'C2');
```

问题三：

建立表 tempstudent，结构与 students 结构相同，其记录均从 student 表获取

实现代码：

```
1 Select * Into tempstudent From student;
```

问题四：

将所有学生的成绩加 5 分

实现代码：

```
1 Update score Set Grade = Grade + 5;
```

问题五：

将姓名为 sue 的学生所在系改为电子信息系

实现代码：

```
1 UPDATE student
2 SET department = '电子信息'
3 WHERE Student_name = 'Sue';
```

问题六：

将选课为 database 的学生成绩加 10 分

实现代码：



```
1 UPDATE score
2 SET Grade = Grade + 10
3 WHERE Course_id IN (SELECT Course_id FROM course WHERE Course_name =
'Database');
```

问题七:

删除所有成绩为空的选修记录

实现代码:

```
1 DELETE
2 FROM score
3 WHERE Grade IS NULL;
```

问题八:

删除学生姓名为 deepa 的学生记录

实现代码:

```
1 DELETE
2 FROM student
3 WHERE Student_name = 'deepa';
```

问题九:

删除计算机系选修成绩不及格的学生的选修记录。

实现代码:

```
1 DELETE
2 FROM student
3 WHERE Student_id IN (SELECT Student_id FROM student s1, score s2 WHERE
s1.Student_id = s2.Student_id AND s1.department = 'Computer' AND
s2.Grade < 60);
```

**【实验四】**

一、简单查询实验

问题一：

查询全体学生的学号、姓名、所在系，并为结果集的各列设置中文名称。

实现代码：

```
1 SELECT Student_id '学号', Student_name '姓名', department '所在系'
2 FROM student;
```

问题二：

查询全体学生的选课情况，并为所有成绩加 5 分。

实现代码：

```
1 SELECT Student_name, Course_name
2 FROM score s1, student s2, course
3 WHERE s1.Student_id = s2.Student_id AND s1.Course_id =
4 course.Course_id;
5
6 UPDATE score
7 SET Grade = Grade + 5;
```

问题三：

显示所有选课学生的学号，去掉重复行。

实现代码：

```
1 SELECT DISTINCT Student_id
2 FROM score;
```

问题四：

查询选课成绩大于 80 分的学生。

实现代码：

```
1 SELECT *
2 FROM score
3 WHERE Grade > 80;
```

问题五：

查询年龄在 20 到 30 之间的学生学号，姓名，所在系

实现代码：

```
1 SELECT Student_id, Student_name, department
2 FROM student
3 WHERE age BETWEEN 20 AND 30;
```

问题六：

查询数学系、计算机系、艺术系的学生学号，姓名。

实现代码：



```
1 SELECT Student_id, Student_name
2 FROM student
3 WHERE department = 'Math' OR department = 'art' OR department =
   'computer';
```

问题七:

查询姓名第二个字符为 u 并且只有 3 个字符的学生学号, 姓名

实现代码:

```
1 SELECT Student_id, Student_name
2 FROM student
3 WHERE Student_name LIKE '_u_';
```

问题八:

查询所有以 S 开头的学生。

实现代码:

```
1 SELECT Student_id, Student_name
2 FROM student
3 WHERE Student_name LIKE 'S%';
```

问题九:

查询姓名不以 S、D、或 J 开头的学生

实现代码:

```
1 SELECT Student_id, Student_name
2 FROM student
3 WHERE Student_name NOT LIKE 'S%' OR Student_name NOT LIKE 'D%' OR
   Student_name NOT LIKE 'J%';
```

问题十:

查询没有考试成绩的学生和相应课程号 (成绩值为空) is null

实现代码:

```
1 SELECT Student_id
2 FROM score
3 WHERE Grade IS NULL;
```

问题十一:

求年龄大于 19 岁的学生的总人数

实现代码:

```
1 SELECT COUNT(*)
2 FROM student
3 WHERE age > 19;
```

问题十二:

分别求选修了 c 语言课程的学生平均成绩、最高分、最低分学生。

实现代码:



```
01 SELECT AVG(Grade), MAX(Grade), MIN(Grade)
02 FROM score, course
03 WHERE score.Course_id = course.Course_id AND course.Course_name =
04 'CProgram';
05
06 SELECT score.Student_id
07 FROM score
08 WHERE score.Grade = (
09     SELECT MAX(Grade)
10     FROM score, course
11     WHERE score.Course_id = course.Course_id AND course.Course_name =
12     'CProgram'
13 ) OR score.Grade = (
14     SELECT MIN(Grade)
15     FROM score, course
16     WHERE score.Course_id = course.Course_id AND course.Course_name =
17     'CProgram'
18 )
```

问题十三:

求学号为 20010102 的学生总成绩

实现代码:

```
1 SELECT SUM(Grade)
2 FROM score
3 WHERE Student_id = '20010102';
```

问题十四:

求每个选课学生的学号, 姓名, 总成绩

实现代码:

```
1 SELECT score.Student_id, Student_name, SUM(Grade)
2 FROM score
3 JOIN student ON score.Student_id = student.Student_id
4 GROUP BY score.Student_id, Student_name;
```

问题十五:

求课程号及相应课程的所有的选课人数

实现代码:

```
1 SELECT Course_id, COUNT(Student_id)
2 FROM score
3 GROUP BY Course_id;
```

问题十六:

查询选修了 3 门以上课程的学生姓名学号

实现代码:



```
1 SELECT student.Student_name, student.Student_id
2 FROM student
3 JOIN score ON student.Student_id = score.Student_id
4 GROUP BY student.Student_name, student.Student_id
5 HAVING COUNT(score.Course_id) > 3;
```

二、多表连接查询

问题一：

查询每个学生基本信息及选课情况

实现代码：

```
1 SELECT student.Student_id, student.Student_name, course.Course_id,
2 course.Course_name
3 FROM student, course, score
4 WHERE student.Student_id = score.Student_id AND course.Course_id =
5 score.Course_id;
```

问题二：

查询每个学生学号姓名及选修的课程名、成绩

实现代码：

```
1 SELECT student.Student_id, student.Student_name, course.Course_name,
2 score.Grade
3 FROM student, score, course
4 WHERE student.Student_id = score.Student_id AND score.Course_id =
5 course.Course_id;
```

问题三：

求计算机系选修课程超过 2 门课的学生学号姓名、平均成绩并按平均成绩降序排列

实现代码：

```
1 SELECT student.Student_id, student.Student_name, AVG(score.Grade)
2 FROM student, score
3 WHERE student.Student_id = score.Student_id AND student.department =
4 'Computer'
5 GROUP BY student.Student_id, student.Student_name
6 HAVING COUNT(score.Course_id) > 2
7 ORDER BY AVG(score.Grade) DESC;
```

问题四：

查询与 sue 在同一个系学习的所有学生的学号姓名

实现代码：

```
1 SELECT Student_id, Student_name
2 FROM student
3 WHERE student.department IN (
4 SELECT department FROM student
5 WHERE Student_name = 'sue'
6 );
```


**问题五:**

查询所有学生的选课情况，要求包括所有选修了课程的学生和没有选课的学生，显示他们的姓名学号课程号和成绩（如果有）

实现代码:

```
1 SELECT student.Student_name, student.Student_id, score.Course_id,  
2    score.Grade  
3 FROM student  
   LEFT JOIN score ON student.Student_id = score.Student_id;
```

**【实验五】**

在已经建立的 studentInfo 数据库的 3 个表基础上，完成下列操作：

问题一：

建立数学系的学生视图；

实现代码：

```
1 CREATE VIEW MathStudent AS
2 SELECT *
3 FROM student
4 WHERE department = 'Math';
```

问题二：

建立计算机系选修了课程名为 database 的学生的视图，视图名为 compStudentview，该视图的列为学号、姓名、成绩

实现代码：

```
1 CREATE VIEW compStudentview AS
2 SELECT
3     s.Student_id AS 学号,
4     s.Student_name AS 姓名,
5     sc.Grade AS 成绩
6 FROM student s
7 JOIN score sc ON s.Student_id = sc.Student_id
8 JOIN course c ON sc.Course_id = c.Course_id
9 WHERE s.department = 'Computer' AND c.Course_name = 'database';
```

问题三：

创建一个名为 studentSumview 的视图，包含所有学生学号和总成绩

实现代码：

```
1 CREATE VIEW studentSumview AS
2 SELECT
3     Student_id AS 学号,
4     SUM(Grade) AS 总成绩
5 FROM score
6 GROUP BY Student_id;
```

问题四：

建立一个计算机系学生选修了课程名为 database 并且成绩大于 80 分的学生视图，视图名为 CompstudentView1，视图的列为学号姓名成绩。

实现代码：



```
01 CREATE VIEW CompstudentView1 AS
02 SELECT
03     s.Student_id AS 学号,
04     s.Student_name AS 姓名,
05     sc.Grade AS 成绩
06 FROM student s
07 JOIN score sc ON s.Student_id = sc.Student_id
08 JOIN course c ON sc.Course_id = c.Course_id
09 WHERE s.department = 'Computer'
10     AND c.Course_name = 'database'
11     AND sc.Grade > 80;
```

问题五:

使用 sql 语句删除 compstudentview1 视图。

实现代码:

```
1 DROP VIEW CompstudentView1;
```

**【实验六】****问题:**

创建对 studentinfo 数据库表 student 进行插入、修改和删除的三个存储过程：insertStudent、updateStudent 和 deleteStudent。

实现代码:

insertStudent

```
01 CREATE PROCEDURE insertStudent
02     @Student_id VARCHAR(20),
03     @Student_name VARCHAR(50),
04     @sex VARCHAR(10),
05     @age INT,
06     @department VARCHAR(50)
07 AS
08 BEGIN
09     INSERT INTO student(Student_id, Student_name, sex, age, department)
10     VALUES(@Student_id, @Student_name, @sex, @age, @department)
11 END;
```

updateStudent

```
01 CREATE PROCEDURE updateStudent
02     @Student_id VARCHAR(20),
03     @Student_name VARCHAR(50),
04     @sex VARCHAR(10),
05     @age INT,
06     @department VARCHAR(50)
07 AS
08 BEGIN
09     UPDATE student
10     SET Student_name = @Student_name,
11         sex = @sex,
12         age = @age,
13         department = @department
14     WHERE Student_id = @Student_id;
15 END;
```

deleteStudent

```
1 CREATE PROCEDURE deleteStudent
2     @Student_id VARCHAR(20)
3 AS
4 BEGIN
5     DELETE FROM student
6     WHERE Student_id = @Student_id;
7 END;
```

**【实验七】****【问题描述】**

本实验所使用的数据库为 sqlite，数据库驱动程序为：sqlite-jdbc-3.7.2.jar。

点击以下链接下载压缩文件，解压后，可在解压目录中找到该驱动程序：

[sqlite-jdbc.rar](#)

在 sqlite 中，假如要连接的数据库的名称为 test.db，则采用 JDBC 连接该数据库的 url 为：

jdbc:sqlite:test.db

本次实验中，数据库名称为 company，数据库中有一张公司员工信息表 EMPLOYEE，创建该表的 sql 语句如下：

```
CREATE TABLE EMPLOYEE(  
  ID          INT PRIMARY KEY NOT NULL,  
  NAME        TEXT NOT NULL,  
  AGE         INT NOT NULL,  
  SALARY      REAL NOT NULL,  
  KPI         CHAR(10) NOT NULL  
)
```

该表中包含多条数据，使用 Java 语言基于 JDBC 编写程序，将 KPI（绩效考核）为 D 的员工的薪水降低 5000 元。

操作完毕后，使用查询语句按照薪水从高到低排序输出所有字段，字段格式如下：

ID, NAME, AGE, SALARY, KPI

ID, NAME, AGE, SALARY, KPI

ID, NAME, AGE, SALARY, KPI

ID, NAME, AGE, SALARY, KPI

每条记录输出一行，行内字段以逗号隔开，除此之外，不添加任何其他格式控制信息。

【输入形式】

company.db 数据库，提示：使用 jdbc 连该数据库

【输出形式】

ID, NAME, AGE, ADDRESS, SALARY, KPI

ID, NAME, AGE, ADDRESS, SALARY, KPI

ID, NAME, AGE, ADDRESS, SALARY, KPI

ID, NAME, AGE, ADDRESS, SALARY, KPI

【样例输入】

company.db 文件

【样例输出】

1, Paul, 36, 8000, A

2, Allen, 27, 7000, B

3, Teddy, 30, 2500, D

【样例说明】

sqlite 数据库使用文件来存储数据库的所有数据，所以输入文件 company.db 为表示公司信息的数据库，在该文件中包含了 EMPLOYEE 表信息。

【解题提示】

注意：sqlite-jdbc-3.7.2.jar 包并没有放在 classpath 中，评测系统在执行提交的 Java 程序时，sqlite-jdbc-3.7.2.jar 包会被拷贝到当前执行程序所在的目录中。因此，在你的程序中，首先需要显式加载该 jar 包并使用以下代码的方式创建连接，样例代码如下：

```
static Connection c; //类属性
```

//创建连接内容

```
ClassLoader cl = new URLClassLoader(new URL[] {new  
File("sqlite-jdbc-3.7.2.jar").toURI().toURL()});  
Class clzz = Class.forName("org.sqlite.JDBC", true, cl);  
Driver d = (Driver)clzz.newInstance();  
DriverManager.registerDriver(d);
```



```
c = d.connect("jdbc:sqlite:company.db", new Properties());  
c.setAutoCommit(false);
```

请务必参考此代码来创建连接返回 Connection 对象或者其他方式使用连接, 如果不使用会造成评测异常情况。

实现代码:

```
package ex7;  
import java.sql.*;  
import java.net.URL;  
import java.net.URLClassLoader;  
import java.io.File;  
import java.util.Properties;  
  
public class main {  
    public static void main(String[] args) {  
        Connection c = null;  
        Statement stmt = null;  
        ResultSet rs = null;  
  
        try {  
            ClassLoader cl = new URLClassLoader(new URL[] {new  
File("sqlite-jdbc-3.7.2.jar").toURI().toURL()});  
            Class clzz = Class.forName("org.sqlite.JDBC", true, cl);  
            Driver d = (Driver) clzz.newInstance();  
            DriverManager.registerDriver(d);  
            c = d.connect("jdbc:sqlite:company.db", new Properties());  
  
            c.setAutoCommit(false);  
  
            stmt = c.createStatement();  
  
            String updateSql = "UPDATE EMPLOYEE SET SALARY = SALARY - 5000 WHERE KPI =  
'D'";  
            stmt.executeUpdate(updateSql);  
  
            c.commit();  
  
            String selectSQL = "SELECT * FROM EMPLOYEE ORDER BY SALARY DESC";  
            rs = stmt.executeQuery(selectSQL);  
  
            while(rs.next()) {  
                int id = rs.getInt("id");  
                String name = rs.getString("NAME");  
                int age = rs.getInt("AGE");  
                double salary = rs.getDouble("SALARY");  
                String kpi = rs.getString("KPI");  
            }  
        }  
    }  
}
```



```
        System.out.println(id + "," + name + "," + age + "," + salary + "," + kpi);
    }
} catch (Exception e) {
    e.printStackTrace();
} finally {
    try {
        if (rs != null) rs.close();
        if (stmt != null) stmt.close();
        if (c != null) c.close();
    } catch (SQLException se) {
        se.printStackTrace();
    }
}
}
```

查询结果:

```
4,Mark,25,60000.0,D
1,Paul,32,20000.0,A
3,Teddy,23,20000.0,C
2,Allen,25,15000.0,B
```

